pointer from its parent.
#11 Return `true`, since the key was removed.

A simpler approach, instead, consists in preemptively pushing down the node to be removed, all the way until it reaches a leaf: as a leaf, it can then be can just disconnected from the tree without any effect.

Conceptually, it's like assigning the lowest possible priority to the node to be removed, and fixing heap's invariants by pushing down the node: the operation won't stop until the node with an infinite (negative) priority will reach a leaf.

This is illustrated in figures 3.10 and 3.11, and described in listings 3.6: in the latter, in particular, we see why it was useful to have method search return the node where the key was found: we can re-use it now to write the remove method, whose first step is indeed searching the key to remove and them, if it was found, the internal delete method will take directly the node that needs to be pushed down.

Special care, as always, need to be paid in case we remove the root.

Let's follow how the algorithm work using our example; suppose we would like to remove "Butter" from our inventory (for instance because we won't sell anymore or we sold all of it).

The first step, shown in figure 3.10.A, is searching for the key "`Butter`" in the tree (line #2 in listing 3.6); once we found the node (that's obviously not `null`, line #3), as usual marked with `X` in the figures, we verify that it's neither the root nor a leaf (hence check at line #4 returns false), so we can enter the while loop at line #5.